CST1500 – Laboratory Worksheet - Week 9

These weekly laboratory worksheets provide information and exercises to complete during class or at your own leisure. The practical exercises will assist you with your coursework as well as provide you with valuable knowledge and skills. Please do your best to keep up with the weekly laboratory worksheets in order to maximise your potential for good marks and satisfactory progress in the module.

Exercises are denoted in red writing. For example:

EX 1: Identification of PC parts (identify as many as you can).

Contents

Lectures videos to watch this week 1

Python Multithreading – Python 3 threading module 1

Synchronizing Threads 7

Lectures videos to watch this week

Lecture 18 IO Systems File Systems
Lecture 19 Security

Python Multithreading – Python 3 threading module

Python Multithreading

Python's threading module allows us to create threads as objects. In this lab session, we shall learn how to work with threads in detail.

This lab session will cover the following:

- Simple Example – Multiple threads
- Create a thread
- Start a thread
- Pass arguments to a thread function
- Is Thread Alive
- Thread Name

Background

Running several threads is similar to running several different programs concurrently, but with the following benefits:

• Multiple threads within a process share the same data space with the main thread and can therefore share information or communicate with each other more easily than if they were separate processes.
• Threads are sometimes called light-weight processes and they do not require much memory overhead; they are (computationally) cheaper than processes.

A thread has a beginning, an execution sequence, and a conclusion. It has an instruction pointer that keeps track of where within its context it is currently running.

• It can be pre-empted (interrupted).
• It can temporarily be put on hold (also known as sleeping) while other threads are running - this is called yielding.

There are two different kind of threads:

• kernel thread
• user thread

Kernel Threads are a part of the operating system, while the user or user-space threads are not implemented in the kernel.

There are two modules which support the usage of threads in Python3:

• _thread
• threading

The thread module has been "deprecated" for quite a long time. Users are encouraged to use the threading module instead. Hence, in Python 3, the module "thread" is not available anymore. However, it has been renamed to "_thread" for backwards compatibility in Python3.

The Threading Module

The threading module exposes all the methods of the thread module and provides some additional methods:

• threading.activeCount() − Returns the number of thread objects that are active.
• threading.currentThread() − Returns the number of thread objects in the caller's thread control.
• threading.enumerate() − Returns a list of all thread objects that are currently active.

In addition to the methods, the threading module has the Thread class that implements threading. The methods provided by the Thread class are as follows:

• run() − The run() method is the entry point for a thread.
• start() − The start() method starts a thread by calling the run method.
• join([time]) − The join() waits for threads to terminate.
• isAlive() − The isAlive() method checks whether a thread is still executing.
• getName() − The getName() method returns the name of a thread.
• setName() − The setName() method sets the name of a thread.

EX 1: Understanding the Threading module in Python

Simple Example of the Threading module

We shall look into a simple example of the threading module, and then look at working with threads in detail.

Note: The following examples are run in an environment with Python3 installed.

The following is a simple example to create multiple threads using the threading module.

```
import threading
def f():
     print('Thread function\n')
     return
for i in range(3):
     t = threading.Thread(target=f)
     t.start()
```

Expected Outcome

```
Thread function
Thread function
Thread function
```

Create a thread using the Threading module

You can create a thread in one of the two following ways:

1. Passing a method to the Thread class constructor.

```
def f():
     print('Thread function\n')
     return
t = threading.Thread(target=f)
```

2. Overriding the run() method in a subclass of threading.Thread.

```
import threading
class CustomThread(threading.Thread):
     def run(self):
          print('Custom thread function.\n')
for i in range(3):
     t = CustomThread()
```

Start a Thread using the Threading module

A thread is started by invoking the start() method on the thread object.

```
import threading
import time
def f():
     print('Thread running.\n')
     return
# start threads by passing function to Thread constructor
for i in range(3):
     t = threading.Thread(target=f)
     t.start()
```

Expected Output

```
Thread running
Thread running
Thread running
```

Passing arguments to the function supplied to Thread

To pass arguments to the function supplied to Thread constructor, pass args in the Thread constructor as shown below:

```
import threading
import time
def f(i):
        for p in range(3):
                time.sleep(i+1)
                print('Thread #',i,"\n")
                time.sleep(i)
        return
# start threads by passing function to Thread constructor
for i in range(3):
        t = threading.Thread(target=f, args=(i,))
        t.start()
```

Expected Output

```
Thread # 0
Thread # 1
Thread # 0
Thread # 2
Thread # 0
Thread # 1
Thread # 1
Thread # 2
Thread # 2
```

EX 2: More examples of understanding the Threading module in Python

Checking if the is Thread Alive!

threading.Thread.is_alive() could be used to check if the thread is alive or not. Thread.is_alive() returns True if the thread is alive, False if not alive.

```
import threading
import time
def f(i):
     time.sleep(i)
     return
# threads
t1 = threading.Thread(target=f, args=(1.2,), name="Thread#1")
t1.start()
t2 = threading.Thread(target=f, args=(2.2,), name="Thread#2")
t2.start()
for p in range(5):
     time.sleep(p*0.5)
     print('[',time.ctime(),']', t1.getName(), t1.is_alive())
     print('[',time.ctime(),']', t2.getName(), t2.is_alive())
```

Expected output

```
[ Wed Dec 11 17:58:54 2019 ] Thread#1 True
[ Wed Dec 11 17:58:54 2019 ] Thread#2 True
[ Wed Dec 11 17:58:55 2019 ] Thread#1 True
[ Wed Dec 11 17:58:55 2019 ] Thread#2 True
[ Wed Dec 11 17:58:56 2019 ] Thread#1 False
[ Wed Dec 11 17:58:56 2019 ] Thread#2 True
[ Wed Dec 11 17:58:57 2019 ] Thread#1 False
[ Wed Dec 11 17:58:57 2019 ] Thread#2 False
[ Wed Dec 11 17:58:59 2019 ] Thread#1 False
[ Wed Dec 11 17:58:59 2019 ] Thread#2 False
```

Show and discuss the above output with your lab tutor, SLA or classmate.

Thread Name

Thread name could be set or read using the setName() and getName() methods. In a function being called inside a thread, to get the current thread, use threading.current_thread(). In the following example we shall use this method to get current thread object.

```
import threading
import time
def f(i):
     for p in range(3):
          time.sleep(i+1.5)
          print(threading.current_thread().getName())
     return
```

```
# start threads by passing function to Thread constructor
for i in range(3):
     t = threading.Thread(target=f, args=(i,))
     t.setName( 'Thread#'+str(i) )
     t.start()
```

Expected Output

```
Thread#0
Thread#1
Thread#0
Thread#2
Thread#0
Thread#1
Thread#2
Thread#1
Thread#2
```

Based on your gained knowledge from the lecture slides, explain to your lab tutor, SLA or classmate when does a Thread stop?

EX 3: Advanced Multithreading programs in Python

Run and explain the obtained output from the following script:

```
import threading
import time

exitFlag = 0

class myThread (threading.Thread):
     def __init__(self, threadID, name, counter):
          threading.Thread.__init__(self)
          self.threadID = threadID
          self.name = name
          self.counter = counter
     def run(self):
          print ("Starting " + self.name)
          print_time(self.name, self.counter, 5)
          print ("Exiting " + self.name)

def print_time(threadName, delay, counter):
     while counter:
          if exitFlag:
               threadName.exit()
          time.sleep(delay)
     print ("%s: %s" % (threadName, time.ctime(time.time())))
     counter -= 1

# Create new threads
thread1 = myThread(1, "Thread-1", 1)
```

```
thread2 = myThread(2, "Thread-2", 2)

# Start new Threads
thread1.start()
thread2.start()
thread1.join()
thread2.join()
print ("Exiting Main Thread")
```

Synchronizing Threads

The threading module provided with Python includes a simple-to-implement locking mechanism that allows you to synchronize threads. A new lock is created by calling the Lock() method, which returns the new lock.

The acquire(blocking) method of the new lock object is used to force the threads to run synchronously. The optional blocking parameter enables you to control whether the thread waits to acquire the lock.

If blocking is set to 0, the thread returns immediately with a 0 value if the lock cannot be acquired and with a 1 if the lock was acquired. If blocking is set to 1, the thread blocks and wait for the lock to be released.

The release() method of the new lock object is used to release the lock when it is no longer required.

EX 4: Understanding thread synchronisation

Understand and Run the following script and explain the obtained results to your lab tutor, SLA or classmate.

```
#!/usr/bin/python3

import threading
import time

class myThread (threading.Thread):
     def __init__(self, threadID, name, counter):
          threading.Thread.__init__(self)
          self.threadID = threadID
          self.name = name
          self.counter = counter
     def run(self):
          print ("Starting " + self.name)
          # Get lock to synchronize threads
          threadLock.acquire()
          print_time(self.name, self.counter, 3)
          # Free lock to release next thread
          threadLock.release()
```

```
def print_time(threadName, delay, counter):
     while counter:
          time.sleep(delay)
          print ("%s: %s" % (threadName,
          time.ctime(time.time())))
          counter -= 1

threadLock = threading.Lock()
threads = []

# Create new threads
thread1 = myThread(1, "Thread-1", 1)
thread2 = myThread(2, "Thread-2", 2)

# Start new Threads
thread1.start()
thread2.start()

# Add threads to thread list
threads.append(thread1)
threads.append(thread2)

# Wait for all threads to complete
for t in threads:
t.join()
print ("Exiting Main Thread")
```

Multithreaded Priority Queue

The Queue module allows you to create a new queue object that can hold a specific number of items. The following methods are used in order to control the Queue:

• get() − The get() removes and returns an item from the queue.
• put() − The put adds an item to a queue.
• qsize() − The qsize() returns the number of items that are currently in the queue.
• empty() − The empty( ) returns True if the queue is empty; otherwise, False.
• full() − the full() returns True if the queue is full; otherwise, False.

EX 5: Understanding the Multithreading priority Queue

Understand and Run the following script and explain the expected output to your lab tutor, SLA or classmate.

```
usr/bin/python3

import queue
import threading
import time

exitFlag = 0
```

```
class myThread (threading.Thread):
     def __init__(self, threadID, name, q):
          threading.Thread.__init__(self)
          self.threadID = threadID
          self.name = name
          self.q = q
     def run(self):
          print ("Starting " + self.name)
          process_data(self.name, self.q)
          print ("Exiting " + self.name)

def process_data(threadName, q):
     while not exitFlag:
          queueLock.acquire()
          if not workQueue.empty():
               data = q.get()
               queueLock.release()
               print ("%s processing %s" % (threadName, data))
          else:
               queueLock.release()
               time.sleep(1)

threadList = ["Thread-1", "Thread-2", "Thread-3"]
nameList = ["One", "Two", "Three", "Four", "Five"]
queueLock = threading.Lock()
workQueue = queue.Queue(10)
threads = []
threadID = 1

# Create new threads
for tName in threadList:
     thread = myThread(threadID, tName, workQueue)
     thread.start()
     threads.append(thread)
     threadID += 1

# Fill the queue
queueLock.acquire()
for word in nameList:
     workQueue.put(word)
queueLock.release()

# Wait for queue to empty
while not workQueue.empty():
     pass

# Notify threads it's time to exit
exitFlag = 1

# Wait for all threads to complete
```

```
for t in threads:
     t.join()
print ("Exiting Main Thread")
```

EX 6: Multithreading Example for Locking

Understand and discuss the obtained output with your lab tutor, SLA or classmate.

```
#Python multithreading example to demonstrate locking.
#1. Define a subclass using threading.Thread class.
#2. Instantiate the subclass and trigger the thread.
#3. Implement locks in thread's run method.

import threading
import datetime

exitFlag = 0

class myThread (threading.Thread):
     def __init__(self, name, counter):
          threading.Thread.__init__(self)
          self.threadID = counter
          self.name = name
          self.counter = counter
     def run(self):
          print("\nStarting " + self.name)
          # Acquire lock to synchronize thread
          threadLock.acquire()
          print_date(self.name, self.counter)
          # Release lock for the next thread
          threadLock.release()
          print("Exiting " + self.name)

def print_date(threadName, counter):
     datefields = []
     today = datetime.date.today()
     datefields.append(today)
     print("{}[{}]: {}".format( threadName, counter,
     datefields[0] ))

threadLock = threading.Lock()
threads = []

# Create new threads
thread1 = myThread("Thread", 1)
thread2 = myThread("Thread", 2)

# Start new Threads
thread1.start()
thread2.start()
# Add threads to thread list
```

```
threads.append(thread1)
threads.append(thread2)

# Wait for all threads to complete
for thread in threads:
thread.join()
print("\nExiting the Program!!!")
```

More details and examples of the multithreading and synchronisation mechanism are available at: https://realpython.com/intro-to-python-threading/